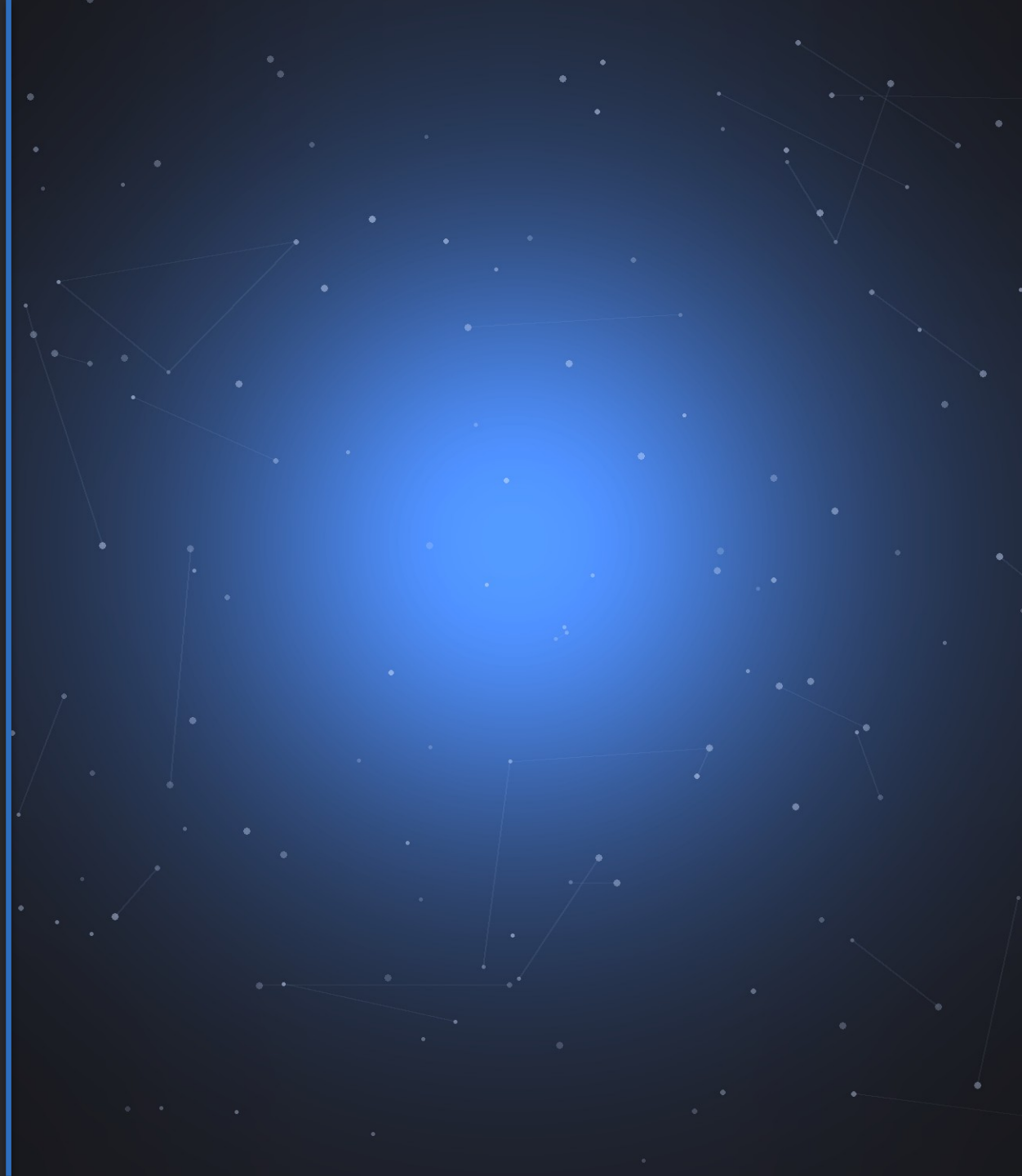

AIA COPILOT TRAINING - DAY 2 OF 3

Claude Code Extensions

Mastronardi Produce | Wed, Aug 19,
2026

Scott Hay



Day 2: teach it your standards

01 Skills

9:00-10:30 - anatomy, triggers, where a skill lives

02 Hooks & commands

10:40-12:10 - payloads, blocking hooks, command frontmatter

03 Advanced skill systems

12:55-2:20 - evals, run variance, A/B benchmarking

04 Plugins & marketplaces

2:30-3:40 - manifests, versioning, publishing governance

05 Close

3:40-4:00 - toolkit review and the Day 3 bridge

LEARNING OBJECTIVES

What you can do by end of day



Write a skill

A SKILL.md with a description that triggers reliably.



Wire a hook

A PostToolUse hook that enforces a rule on every action.



Build a human-invoked skill

An explicit /workflow with \$ARGUMENTS and model invocation disabled.



Prove it works

Eval loop plus A/B benchmark against raw Claude.



Ship it

Package the toolkit as a plugin on a team marketplace.

THE THREE-DAY ARC

From assistant to engineering system

Day 2 is where Claude stops being generic and starts being yours.



Day 1 - drive one agent well - loop, context, memory



Day 2 - teach it your standards - skills, hooks, plugins



Day 3 - multiply it - MCP, subagents, CI/CD



BLOCK 1 - 90 MINUTES

Skills

Domain expertise Claude loads on demand.

BLOCK 1 - CONCEPT

What a skill actually is

A folder with a playbook - not a magic prompt.



SKILL.md - frontmatter trigger + markdown
playbook



The folder - scripts/, references/, templates
alongside



The trigger - description matching loads it on
demand

Progressive disclosure



Scan

descriptions only



Match

prompt fits trigger



Load body

playbook enters
context



Reach deeper

references on
demand

Frontmatter: the current fields



The name field

Kebab-case identifier - becomes the /slash invocation.



The description field

The trigger. Claude reads this to decide when to load.



disable-model-invocation

Only the user can invoke - Claude never auto-loads it.



allowed-tools

Least privilege - a review skill gets Read, Grep, Glob only.



Body target: 1,500-2,000 words of process. Push bulk knowledge into references/ - point, don't dump.

Pushy descriptions win

Claude is conservative - vague descriptions never fire.



Be explicit

List 5-7 trigger phrases users actually type.



Set boundaries

Add 'do NOT use for...' negative triggers.



Test 5 phrasings

If 3+ fail to fire, rewrite the description.



Trigger failure is the #1 reason skills never fire - the description is the single most important line.

Paste this: your first skill

What you are practicing

- Scoping one skill to one job
- A description with trigger phrases
- Least-privilege allowed-tools
- Saying 'current project directory'

PASTE THIS

1

Create a skill at `.claude/skills/code-review/SKILL.md` in the current project directory. Trigger on 'review the code', 'check for bugs', 'QA this file'. Review for security, error handling, and test coverage. Read-only tools.

2

Show me the SKILL.md you created and explain what each frontmatter field does.

Instructions or scripts?



Instructions (SKILL.md)

- Judgment, language, formatting
- Rewrite in brand voice, categorize
- Claude adapts to context



Scripts (scripts/)

- Computation, parsing, file ops
- Deterministic, repeatable output
- Claude runs them, not rewrites them

The one-level-deep rule

01

**Body =
steps**

Process
instructions only.

>

02

references/

Checklists and
templates.

>

03

Never nest

References can't
link deeper.

>

04

Compress

Extract only
what's needed.

Design principles that hold up



Single responsibility

One skill, one job - never review AND refactor AND document.



Build a gotchas section

Log Claude's failure points over time - highest-signal content.



Don't state the obvious

Only include what pushes Claude OUT of its defaults.



Don't railroad

Give goals and constraints, not micromanaged steps.



Run it 5 times

Inconsistent output means ambiguous instructions.

The 9 skill types in the wild

Anthropic catalogued hundreds of internal skills - they cluster into three families.



Knowledge

Library/API reference, domain patterns, docs generation.



Verification

Output checking, product verification, eval harnesses.



Workflow

Formatting, automation, review pipelines, reporting.



Verification skills repay a full engineering week - they make every other skill's output trustworthy.

One skill, every surface

SURFACE	HOW YOUR SKILL LOADS
Claude Code	<code>.claude/skills/</code> scanned automatically at session start
Claude.ai	Same SKILL.md format uploaded to the web app
Agent SDK	Skills folder wired into your custom agents
Plugins	Bundled and installed from a marketplace

Where a skill lives matters

Same file, three homes - each with a different blast radius.



Personal

~/.claude/skills - follows YOU into every project.



Project

.claude/skills - in git, the whole team gets it on clone.



Plugin

Installed from a marketplace - versioned, read-only, many repos.



Prototype personal, promote to project when the team needs it, ship as a plugin when many repos do.

When a skill won't load



Description too vague

No keyword overlap with the prompt - sharpen the triggers.



Wrong location

Must be `.claude/skills/<name>/SKILL.md` - not root, not nested.



Context nearly full

Skills may not load late in a bloated session - check `/context`.



Watch the matching

`claude --debug` shows which skills were considered and why.



Listing over budget

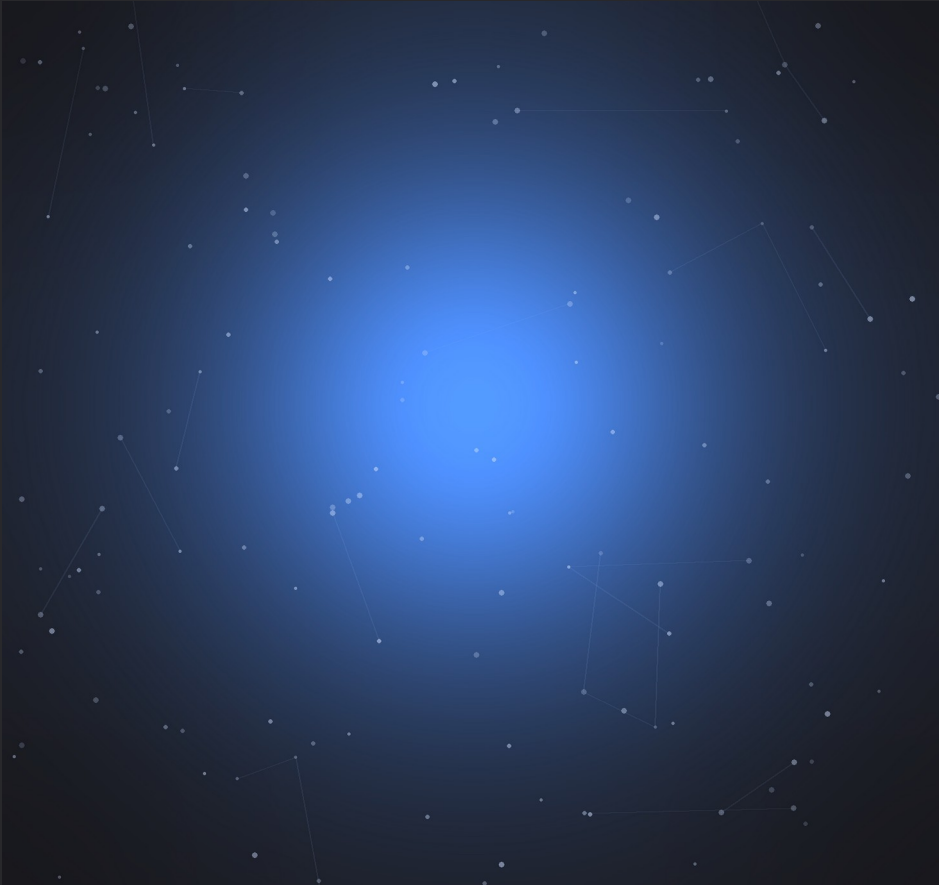
Many skills installed? The name+description listing has a context budget - least-used descriptions get dropped.



Skill listing budget = $\sim 1\%$ of context. Over it, least-used DESCRIPTIONS drop - names stay, auto-trigger dies.

LIVE DEMO

Demo: build a skill, watch it fire



WHAT WE'LL RUN TOGETHER

- 1** Create code-review skill via one prompt
- 2** Trigger it: 'review src/routes/tasks.js'
- 3** Break it: vague description, then /doctor (optional - skim only the unused-skill lines)

RECAP

Three lines to keep

If they remember nothing else from Block 1, they remember these.

“

A skill is an SOP for an AI agent - process in the body, knowledge in references.

“

The description is the trigger. Vague descriptions never fire.

“

Encode only what pushes Claude out of its defaults.

KNOWLEDGE CHECK

Block 1 review

QUESTION	THINK ABOUT...
Two parts of every SKILL.md?	Trigger and playbook
Which field drives auto-invocation?	Where trigger phrases live
What stops Claude auto-loading a skill?	A frontmatter field
State the one-level-deep rule	What nesting breaks

Lab 2A: Build Your First Skill

Hands-on: your own skill in your toolkit repo.



Deliverable

A code-review skill: SKILL.md
+ references/ checklist.



Timebox

30 minutes - if behind, ship
the skill without references.



Success

Fires on 3 phrasings; /doctor
(optional - skim only the
unused-skill lines) shows it
used; committed to git.

BREAK

**Back in 10 - hooks are
next**



BLOCK 2 - 90 MINUTES

Hooks & Human-Invoked Skills

Deterministic enforcement plus explicit, reusable workflows.

BLOCK 2 - CONCEPT

What hooks actually are

Deterministic scripts wrapped around Claude's actions.



Observe - log and audit everything Claude does



Enforce - block dangerous actions before they run



Automate - format, test, notify after every change

How a hook fires



Event

one of 31 lifecycle points



Matcher

regex picks the tool



Handler

your script runs



Exit code

0 allows, 2 blocks

The payload: JSON on stdin

FIELD	WHAT IT TELLS YOUR SCRIPT
session_id / cwd	Which session fired, from which working directory
hook_event_name	Which lifecycle event triggered the handler
tool_name	The tool Claude is using - Write, Edit, Bash...
tool_input	The exact arguments: command, file_path, content
tool_response	What the tool returned - PostToolUse only
(verify at delivery)	Field names re-checked against the hooks reference at the morning-of gate

The event map worth knowing



**PreToolUse /
PostToolUse**

The workhorses - validate before, react after. 95% of use cases.



**SubagentStart /
SubagentStop
TeammateIdle /**

Wrap delegated work - scope rules per subagent (Day 3).



**TaskCreated /
TaskCompleted**

Team and task lifecycle signals for orchestration.



**WorktreeCreate /
WorktreeRemove**

React when parallel worktrees appear and vanish.



**PreCompact /
PostCompact**

Save or restore state around context compaction.



**PermissionRequest
/ PermissionDenied**

Route, log, or auto-triage permission prompts.



31 events in the current hook reference; learn the high-value pairs and use the handout for the full map.

Five handler types



command

Shell script - JSON on stdin, exit code decides. The classic.



HTTP

POST the event to an endpoint - central team telemetry.



MCP tool

Invoke an MCP server tool as the handler.



prompt

Ask a model to judge - non-deterministic, use with care.



agent

Run an agentic verifier with tool access for multi-step judgment.



prompt handlers reintroduce model judgment - keep hard security rules in command handlers with exit codes.

Advisory or blocking?



Advisory (exit 0)

- Logs, warns, formats - never stops
- Message shown, action proceeds
- Start here: observe before you enforce



Blocking (exit 2)

- Action stops before it runs
- stderr reason goes back to Claude
- The agent reads it and adjusts course



Exit semantics (0 allow / 2 block) per current docs - re-verify at the morning-of delivery gate.

Matchers and where hooks live



Matcher = tool filter

Regex on the tool name: Write|Edit. Empty matcher = every tool.



User-level

~/.claude/settings.json - follows YOU across all projects.



Project-level

.claude/settings.json - travels with the repo via git.



Plugin-delivered

Ships inside an installed plugin - fleet-wide enforcement.



Roll out narrow: one matcher, one project. Widen to user or plugin scope only once the hook has earned it.

Pick the right event

YOU NEED TO...	USE THIS EVENT
Block a dangerous command	PreToolUse (exit 2 blocks)
Auto-format after edits	PostToolUse on Write Edit
Load context at session open	SessionStart
Save a summary at session end	Stop / SessionEnd
Audit permission prompts	PermissionRequest

Three hooks every team ships

Copy these - they cover audit, hygiene, and visibility.



Audit log

PostToolUse, empty matcher - timestamp, tool, file to a log.



Auto-format

PostToolUse on Write|Edit - prettier/black by extension.



Notification

Bash matcher - webhook to Slack/Teams on deploy or push.



Handlers read JSON from stdin: `json=$(cat)`, then `jq -r '.tool_name'` - never env vars.

Fail-safe hook discipline

01

Fast

Under 1 second,
always.

>

02

Focused

One hook, one
concern.

>

03

Fail open

Exit 0 on
unknown tools.

>

04

stderr only

Messages never
on stdout.

BLOCK 2 - ARCHITECTURE

The three-layer stack

Each layer covers the failure mode of the one above.



CLAUDE.md - the contract - context and conventions



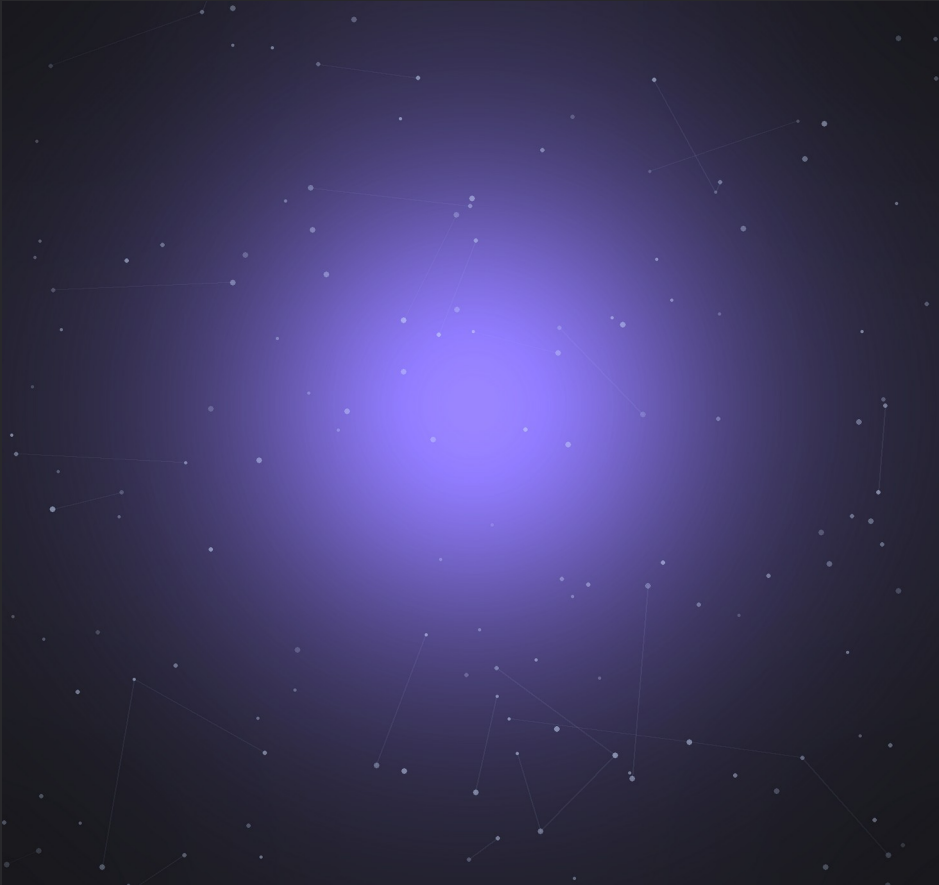
Skills - the execution path - how work gets done



Hooks - the hard boundary - rules that cannot drift

LIVE DEMO

Demo: audit-log hook live



WHAT WE'LL RUN TOGETHER

- 1** Write audit-log.sh, wire settings.json
- 2** Have Claude edit a file - watch the log
- 3** Add a PreToolUse block, trigger it

Custom commands are skills now

The legacy commands folder still works; skills are the recommended form.



Recommended

`.claude/skills/review/SKILL.md`
becomes `/review`.



Legacy compatible

`.claude/commands/review.md`
still works, but has fewer
features.



Human-only when needed

`disable-model-invocation:`
`true` keeps deploy/release
skills explicit.

FOLLOW ALONG

Paste this: a human-invoked `/test-this` skill

What you are practicing

- SKILL.md with frontmatter
- `disable-model-invocation: true`
- `$ARGUMENTS` as the file path

PASTE THIS

1

Create `.claude/skills/test-this/SKILL.md`:
name `test-this`; `disable-model-invocation: true`; tools `Read`, `Write`, `Edit`, `Bash`. Body: write and run tests for `$ARGUMENTS`, covering happy path, edges, and errors.

2

`/test-this src/utils/helpers.js`

Arguments and injection



\$ARGUMENTS

Everything typed after the /command, as one string.



\$1, \$2 positional

Split parameters: /fix-issue 429 high -> \$1=429, \$2=high.



@file references

Pull a named file's content straight into the prompt.



!`cmd` injection

Runs a shell command; its output lands in the prompt first.



Syntax per 2026-07 docs - re-verify at delivery; treat injected shell output as untrusted input.

Command frontmatter controls

Three fields that decide what a command may touch and what it costs.



allowed-tools

Least privilege - a /review command gets read-only tools.



model

Pin a fast, cheap model for mechanical commands.



Human-only gate

disable-model-invocation for deploy and release.



Field names (allowed-tools, model) per 2026-07 skills docs - re-verify at the delivery gate.

Four human-invoked skills worth stealing



/test-this

Generate tests for a file - happy path, edges, errors.



/review

Security, performance, and style pass with actionable fixes.



/deploy-check

Pre-deploy checklist: tests, no console.log, changelog.



/onboard

Analyze a new codebase - stack, architecture, entry points.



A coordinating skill can invoke specialized skills and compile one report.

Command or skill?



Stays a command

- One prompt file, no supporting assets
- You always invoke it yourself
- Cheap to write, trivial to read



Promote to a skill

- Needs scripts, references, templates
- Claude should find it on its own
- Shared, versioned, eval-tested



.claude/commands/git/pr.md invokes as /git:pr - subfolder namespacing keeps a 30-command library findable (verify at delivery).

Human-invoked or model-invoked?



Human-invoked skill

- `disable-model-invocation: true`
- Explicit `/name` invocation
- `$ARGUMENTS` for parameters



Model-invoked skill

- Claude may load it when relevant
- Description matching is critical
- Reusable context or safe workflows

KNOWLEDGE CHECK

Block 2 review

QUESTION	THINK ABOUT...
What do exit codes 0 and 2 mean?	Advisory vs blocking
How does a hook receive its data?	Not environment variables
Name two of the five handler types	What runs when the event fires
Command vs skill - who decides when?	Intent and invocation

Lab 2B: Hooks and Human-Invoked Skills

Hands-on: enforcement wired into your toolkit repo.



Deliverable

Format hook, advisory-then-blocking secret hook, matcher scoping, /review skill.



Timebox

30 minutes - if behind, stop after the blocking hook.



Success

Blocked write triggers a visible self-correction; matcher gap found and closed; committed.

LUNCH

Lunch - back in 45



BLOCK 3 - 85 MINUTES

Advanced Skill Systems

From skills that exist to skills that are proven.

The skill maturity model

01

L1: Prompt

Repeated from memory.

>

02

L2: Skill

SKILL.md + refs, in git.

>

03

L3: System

Evals, hooks, monitoring.

Uplift or encoded preference?



Capability uplift

- Teaches Claude something it can't do
- May expire as models improve
- Test: does base model now match it?



Encoded preference

- YOUR workflow, sequenced YOUR way
- Survives model upgrades
- Test: does it still match your process?

Three patterns for skill systems

A pile of skills makes YOU the glue. Systems chain them.



Shared context

Every skill reads the same CLAUDE.md and rules.



Output as input

One skill writes structured files, the next reads them.



Scheduled

Hooks and CI trigger skill chains automatically.



Files are the API between skills - loose coupling, inspectable at every step.

Skills that carry hooks



/careful

Invoked skill registers PreToolUse blocks on rm -rf, force-push, DROP.



/freeze

Blocks edits outside one directory during focused debugging.



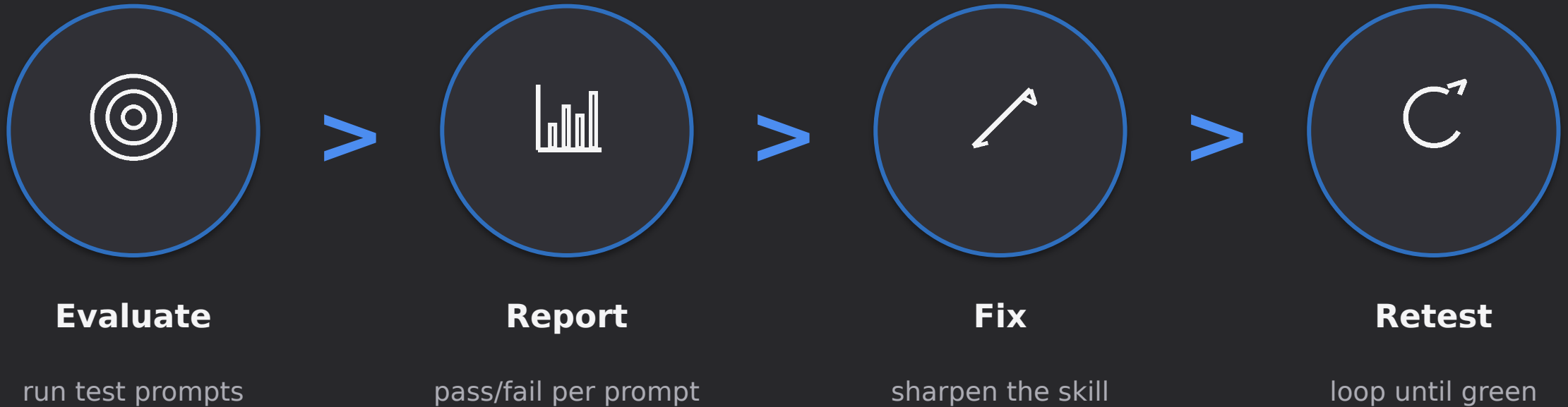
Usage metering

A PreToolUse hook logs which skills fire - finds under-triggering.



These hooks are session-scoped - they activate when the skill is invoked and vanish at session end.

The skill testing loop



Symptom to fix, fast



Never activates

Add 5+ trigger phrases to the description.



Wrong skill fires

Add 'do NOT use for...' boundaries.



Close but not right

Replace vague instructions with testable ones.



Fails on real data

Add edge-case handling for messy input.



Adds extra commentary

Add an 'output ONLY X' constraint.

A/B: is your skill helping?

Same prompts, twice - with the skill and without.



With skill

Your instructions steer the output.



Baseline

Raw Claude - improves with every model release.



Blind judge

A separate agent picks the winner, unbiased.



If baseline wins, retire the skill - an outdated skill can hold a newer model back.

One run is an anecdote

01

Run it 5x

Same eval, five times.



02

See the spread

6/8, 7/8, 5/8 - normal.



03

Use the band

Report the min-max.



04

Then compare

Trust gaps beyond the band.

BLOCK 3 - TOOLING

skill-creator: the meta-skill

Anthropic's official plugin - use AI to build and test AI automation.



Create - scaffold a well-structured skill



Improve - tune descriptions that under-trigger



Evaluate - write and run evals, no code



Benchmark - pass rate, time, tokens vs baseline

Paste this: eval your skill

What you are practicing

- Should-trigger and should-not lists
- Recording pass/fail honestly
- Description fixes from failures

PASTE THIS

1

Create an eval set for my code-review skill: 5 prompts that SHOULD trigger it and 3 that should NOT. Run each, report which fired correctly as a pass/fail table.

2

Analyze the failures and propose one improved description. Show old vs new side by side.

Skills become specifications

Where Anthropic says this is going.

“

Today a SKILL.md tells Claude HOW - step-by-step procedure.

“

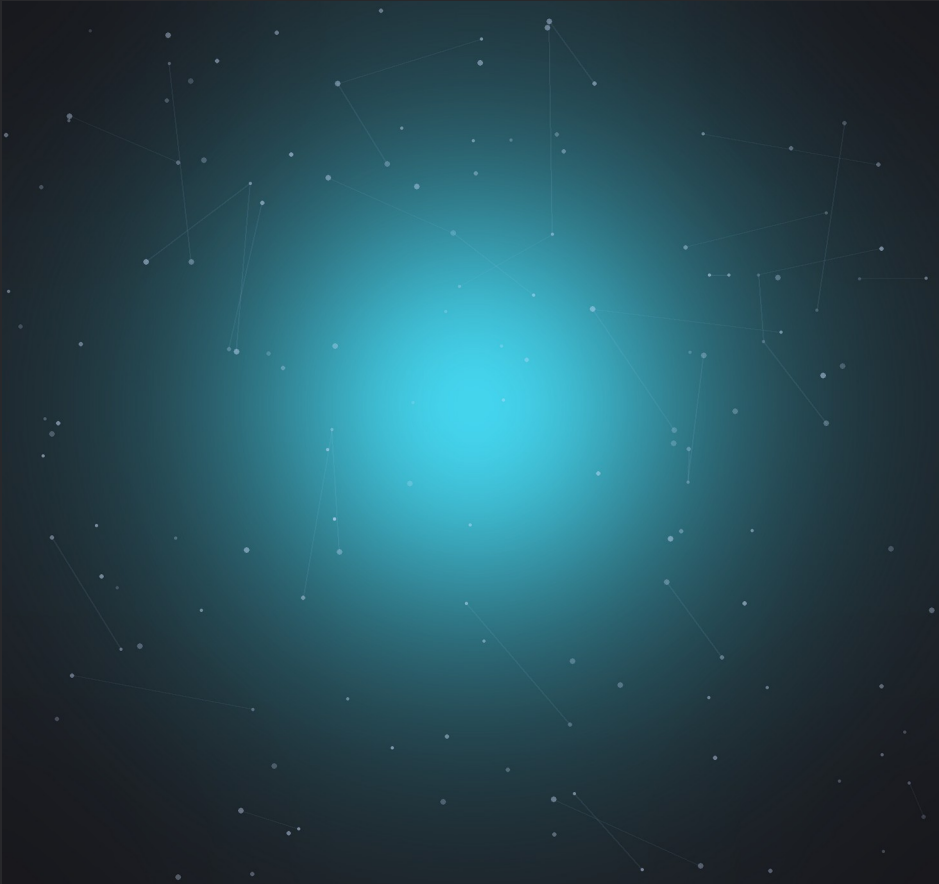
Tomorrow it may only state WHAT - the model finds the how.

“

Your evals already describe the what. They are future-proof.

LIVE DEMO

Demo: eval and benchmark live



WHAT WE'LL RUN TOGETHER

- 1** Run the eval set - see pass/fail
- 2** Benchmark: skill vs raw baseline
- 3** Fix the description, rerun, compare

KNOWLEDGE CHECK

Block 3 review

QUESTION	THINK ABOUT...
Name the three maturity levels	Where value compounds
Uplift vs preference - which expires?	What model releases change
What makes the A/B judge honest?	Bias removal
What are files, in a skill system?	The interface between skills

Lab 2C: Advanced Skill Workflows

Hands-on: prove your skill earns its place.



Deliverable

Eval set (5+5), a 5-run variance band, benchmark verdict, improved description.



Timebox

30 minutes - if behind, run evals and the variance band, skip the benchmark.



Success

Variance band recorded; verdict uses the band, not a single run; all in the repo.

BREAK

**Back in 10 - then we
ship it**



BLOCK 4 - 70 MINUTES

Plugins & Marketplaces

Package your standards and ship them to the team.

BLOCK 4 - CONCEPT

A plugin bundles your extension stack

Everything from today, packaged as one installable unit.



Skills & commands - recommended skills plus legacy commands



Agents - specialized subagents, namespaced



Hooks - event-driven enforcement on enable



MCP & LSP - external tools and code intelligence

MCP in one slide (Day 3 deep)

The protocol that connects Claude to external tools.



One standard

Build one server, any MCP client can use it.



Config

.mcp.json at project root;
/mcp verifies connection.



In plugins

Plugins can ship
preconfigured MCP servers.



Tool search defers MCP tool schemas by default; inspect server instructions, data access, and any alwaysLoad exceptions.

Inspect before you install



Publisher and provenance

Who owns it, where is the source, and is the release maintained?



Components

List skills, agents, hooks, scripts, MCP/LSP servers, and dependencies.



Authority

What can execute, which credentials it inherits, and which files or commands it can touch?



Data flow

What leaves the machine, which provider receives it, and where state is stored?



Lifecycle

Version pinning, auto-update behavior, disable/uninstall path, and rollback.



A marketplace is a catalog, not a security boundary. “Official marketplace” does not make every publisher Anthropic.

From browse to running



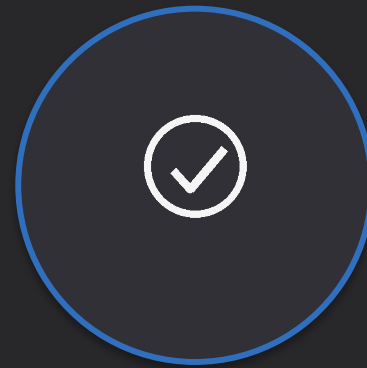
Browse

/plugin Discover tab



Install

name@marketplace



Verify

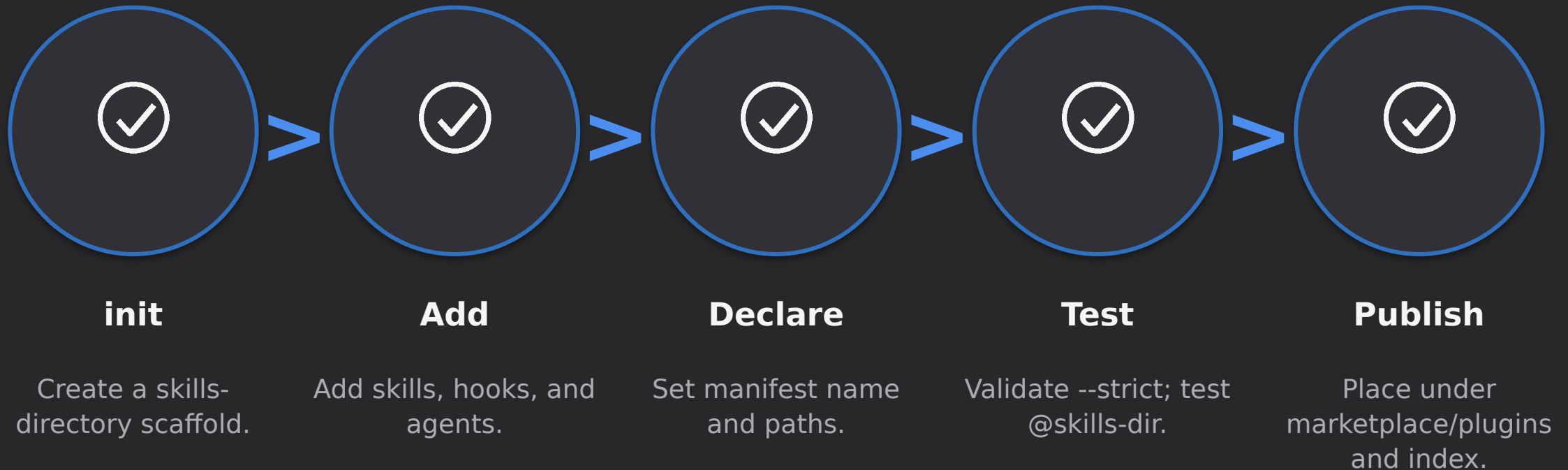
/plugin list or claude
plugin list



Use

namespaced
commands

Author a plugin in five steps



The plugin trust hierarchy



**Built-in or
Anthropic-
maintained**

Highest default confidence, but still confirm the exact component and version.



**Approved internal
marketplace**

Organization-curated, versioned, and governed for your environment.



**Official
marketplace, third-
party publisher**

Discoverable through Anthropic; publisher and maintenance can still be external.



**Any external
marketplace or
direct source**

Untrusted until source, data flow, permissions, and updates are reviewed.



Plugins can execute code, hooks, commands, and MCP/LSP servers with your user privileges. Treat them like executable dependencies.

Plugin estate controls

Inventory, deactivate, and remove without guessing.



claude plugin list/details

See installed state, scope,
and contributed components.



/plugin disable

Stop loading a plugin while
preserving the installation.



/plugin uninstall

Remove it at the intended
user, project, or local scope.



Refresh catalog metadata with /plugin marketplace update; apply changes with /reload-plugins when the install summary requests it.

Paste this: package your toolkit

What you are practicing

- claude plugin init scaffolding
- Manifest that maps your assets
- Local install as the smoke test

PASTE THIS

1

Run claude plugin init dashboard-toolkit --with skills hooks. Move the reviewed skills and hooks into the scaffold; update the manifest.

2

Validate --strict. Show the component inventory, executable code, credentials, network destinations, and rollback path.

Project config or plugin?



Project config (.claude/)

- One repo's standards
- Travels via git clone
- Editable by the whole team



Plugin (.claude-plugin/)

- Standards across MANY repos
- Installed from a marketplace
- Read-only - versioned centrally

Safe change or breaking change?

✓ Safe (minor / patch)

- Add a new skill or command
- Sharpen a description, fix docs
- Bug-fix a hook, same behavior

⚠ Breaking (major)

- Rename or remove a command
- A hook now blocks what it allowed
- New credential or tool scope



Consumers update on your schedule, not theirs - a renamed command breaks every teammate's scripts overnight.

Publishing to the internal catalog

01

Propose

PR against
the
marketplace
repo.

>

02

Review

Security
checklist,
two eyes.

>

03

Version

Semver
bump plus
changelog.

>

04

Announce

Changelog
to the team
channel.

>

05

Deprecate

Sunset path
- never
delete.

LIVE DEMO

Demo: marketplace to running



WHAT WE'LL RUN TOGETHER

- 1** Install skill-creator from the official marketplace
- 2** Inspect with `/plugin list` and claude plugin details
- 3** Validate and publish dashboard-toolkit to a local team marketplace

KNOWLEDGE CHECK

Block 4 review

QUESTION	THINK ABOUT...
What core components can a plugin contribute?	The day's whole stack
Where does the manifest live?	Not the project root
What makes a git repo a marketplace?	Less than you think
Why treat plugins like dependencies?	What they can run

LAB 2D - 30 MINUTES

Lab 2D: Package and Ship

Hands-on: your toolkit, on a real team marketplace.



Deliverable

my-toolkit plugin published to the class marketplace repo, installed by a peer.



Timebox

30 minutes - if behind, local-path install counts.



Success

A classmate installs YOUR plugin and runs your /command.



CLOSE - 20 MINUTES

Close: Your Toolkit

What you built, and the bridge to Day 3.

Your complete extension stack



Skills

Domain playbooks, tested with evals - Lab 2A + 2C.



Hooks

Deterministic enforcement across 31 events - Lab 2B.



Invocation controls

Human-invoked skills via /slash - Lab 2B.



Plugins

All of it packaged, published, peer-installed - Lab 2D.



MCP

External tools, context-cost aware - Day 3 goes deep.

Three principles to keep

The Day 2 engineering mindset in three lines.

“

Encode, don't repeat -
said 3x means
CLAUDE.md, did 3x means
a command, checked 3x
means a hook.

“

A skill without evals is a
hope, not an asset.

“

Systems beat collections -
files are the API between
skills.

Homework: extend your toolkit

Three small artifacts before Day 3 - all in your real work.



One skill

For your actual project, with references, eval-tested.



One human-invoked skill

A daily /workflow with explicit invocation control.



One hook

A safety rule, committed to git.



Bring all three tomorrow - Day 3 wires them into subagents and CI.

CLOSE - DAY 3 BRIDGE

Skills + subagents: the multiplier

Tomorrow your standards stop being loaded - they get delegated.



Today - skills load into one session on demand



Tomorrow - subagents run your skills in parallel, isolated



The payoff - a security-review agent per PR, automatically

END OF DAY 2

You taught it your standards

What you shipped

Two tested skills, two hooks, and a validated plugin a classmate installed from a team marketplace. All committed, all portable.

Tomorrow

Day 3 multiplies it: MCP in depth, subagents and agent teams running your skills in parallel, and headless CI/CD with cost controls.